PS-001

DevPilot: The Mystic Mikado's Autonomous AI Engineering Guild

Background

When a decree arrives in the Emperor's kingdom, his engineering guild of strategists, builders, testers, and guardians must plan, build, verify, and review the change, and adapt when things go wrong. Build DevPilot, a multi-agent AI system that turns a natural-language software request into a working, tested, and reviewed change to an existing small repository.

The core principle is: *Plan like a Shogun. Build like a Samurai. Test like a Sentinel. Adapt until the mission is complete.*

Tasks

A) The Shogun's Council: Planning

- **Requirement Analysis:** Convert the request into objectives and acceptance criteria.
- **Codebase Reconnaissance:** Inspect the project to find relevant files, conventions, and risks.
- **Task Decomposition and Delegation:** Break the mission into tasks, assign them to agents, and revise the plan when new information or blockers appear.
- **Shared Mission State:** Keep a common record of task status, findings, decisions, and open issues across agents.

B) The Samurai Forge: Implementation

- **Tool Selection:** Developer agents choose among at least three tools (e.g., file read/search, file edit, terminal commands).
- **Context-Aware Development:** Follow the project's existing style and structure rather than blindly generating new code.
- **Adaptive Problem Solving:** Investigate unexpected errors and change approach when needed.
- **Controlled Execution:** Restrict commands and file access to the repo folder, and block destructive operations.

C) The Sentinel Dojo: Testing and Verification

- **Automated Testing:** Run the project's tests (or build/lint checks).
- **Failure Investigation and Repair:** Analyze errors, send findings back to the developer agent, and rerun.
- **Independent Verification:** A separate verifier agent or process checks the acceptance criteria.

- **Evidence-Based Completion:** Record what changed, which checks passed or failed, and what remains unresolved. The system must not claim success without evidence.

D) The Shadow Council: Review and Security

- **Code Review:** Review the changes for correctness, readability, consistency, and regressions.
- **Security Inspection:** Run basic checks for exposed secrets, unsafe operations, and injection risks.
- **Cross-Agent Critique:** Reviewers can send findings back to developers. Disagreements are settled by evidence or escalated to a human.
- **Human Oversight:** Require human approval before consequential actions such as merging or applying changes.

E) The Living Campaign: Coordination and Adaptive Autonomy

- **Inter-Agent Communication:** Agents exchange findings and use each other's results. Sequential LLM calls with different personas do not count.
- **Feedback Loops:** Failures or review findings must trigger further investigation and revised actions.
- **Adaptive Orchestration:** Decide when to continue, retry, ask for clarification, or stop.
- **Bounded Autonomy:** Define success conditions, iteration limits, permissions, and escalation rules. Handle failed tools and repeated failures without endless loops.

The architecture, number of agents, and division of responsibilities are up to you.

Technical Requirements

Use any language, framework, AI model, or agent framework. Existing models, libraries, APIs, and tools are allowed. A local repository is enough, and GitHub is not required. The goal is not to maximize the number of agents or tools, but to show that coordination and iterative reasoning contribute meaningfully to the result.

Evaluation Scenarios

Test the system on small controlled repositories covering:

- **Feature Implementation:** Add a feature while preserving existing behavior.
- **Bug Investigation:** Fix a seeded defect using repository context and test output.
- **Test-Driven Recovery:** Repair a repository that has a failing test.
- **Security and Code Review:** Detect and address a planted vulnerability or quality issue.
- **Unexpected Conditions:** Handle an unavailable tool, an ambiguous requirement, or repeated test failures.

Demonstrate at least one complete end-to-end mission and one feedback loop in which new evidence changes what an agent does next.

Evaluation Criteria

- **Task Completion:** Does the change satisfy the acceptance criteria?
- **Agent Coordination:** Do agents share findings and respond to each other's results?
- **Tool-Calling Intelligence:** Are tools chosen appropriately for the situation?
- **Adaptability:** Can the system recover from failures and revise its plan?
- **Code Quality:** Are the changes correct, maintainable, and consistent with the project?
- **Testing and Verification:** Are results backed by real test runs and independent checks?
- **Security and Reliability:** Are permissions, boundaries, and failure conditions handled responsibly?
- **Transparency:** Can users inspect the plan, agent decisions, tool use, and test results?

All reported results must come from actual test runs. Clearly separate verified outcomes from assumptions and model claims.

Deliverables

- Source code and a working demonstration
- Multi-agent architecture with defined agent roles
- Working implementation, testing, and verification workflow
- At least one end-to-end mission with a run log showing a feedback loop
- Setup instructions, configuration files, and a list of supported tools

Documentation (README.md)

Include:

- Architecture diagram and agent responsibilities
- Agent communication, orchestration, and shared-state design
- Setup instructions and configuration
- Available tools and execution boundaries
- Human approval mechanism, security assumptions, and known limitations

Bonus Points

- Parallel or dynamically created agents
- Automatic reruns and self-correction when tests fail
- Persistent memory, or comparison against a single-agent baseline

Expected Outcome

A functional multi-agent system that interprets a development request, plans the work, coordinates agents, uses tools, implements changes, responds to failures, and verifies the result. It should not be a fixed pipeline of chatbots. Let the guild prove its worth not by the number of agents it commands, but by the quality of the software it delivers.

PS-002

Tsukuyomi Watch: The Samurai's Celestial Guide

Background and Description

In feudal Japan, samurai and travelers read the heavens to time their battles, journeys, and rituals. The moon phases, falling stars, and the movement of constellations guided every decision. Today, finding a clear night sky and knowing when celestial events occur is still hard: weather, cloud cover, light pollution, and scattered event sources all get in the way. Meanwhile, the images we capture of the cosmos from Hubble, JWST, and ground telescopes are often clouded by noise and blur, like mist hiding a mountain.

Participants must build a **web platform that acts as a modern warrior's celestial guide**. It should help users find the best stargazing spots, track cosmic events, and restore and explore astronomical images with AI, all wrapped in a Japanese-inspired, mystic interface.

Key Components

1. The Watchtower: Stargazing Location Finder

- Fetch real-time weather data (cloud cover, humidity, visibility) from open APIs.
- Calculate a **"Night Honor Score"** (the visibility score), optionally including a light pollution indicator.
- Show top nearby spots on an interactive map, styled like an ancient scroll map with shrines, mountains, and temples as markers.

2. The Celestial Scroll: Space Event Tracker

- List meteor showers, eclipses, **moon phases**, ISS passes, and rocket launches.

  (Participants can use APIs provided by Open Meteo, NASA etc.)

- Filter by event type and date, using a calendar that follows the **traditional lunar calendar** style.

- Add an alert feature such as “The Moon Rises Tonight at 8:14 PM.”

3. The Spirit Lens: AI Image Restoration

- Let users upload a space image and clean it up (less noise, less blur) using an AI model. Pre-trained models are allowed.
- Do not add fake details. The result must stay true to the original.
- Detect stars and galaxies in the image and show basic info such as brightness and size.

4. The Dojo Dashboard: Visualization Interface

- Interactive **before/after slider** for original vs. restored images, with zoom and overlays for detected objects.

Theme Requirements

- Japanese-inspired UI (ink-wash style, torii gate, cherry blossom, katana motifs, red, black, and gold palette).
- Moon-centric design elements (moon phase tracker, night-mode theme).

Bonus Challenges

- Integrate live telescope data via NASA SkyView API.
- Add spectral/time-domain analysis for transient event detection.
- Gamify with an “Honor Points” system for observing events.

Expected Solution

A complete web app that shows real-time stargazing conditions, curates space events, enhances and analyzes astronomical images with AI, and presents everything in an immersive, samurai-and-moon-themed interface.

PS-003

Small Web Reader: Anonymous Discovery for Independent Blogs

Background:
Most people read the web through algorithmic feeds that decide what they see. RSS and Atom are the older alternative: you follow blogs directly, with no ranking, ads or tracking. The weak spot is discovery, since feed readers assume you already know which blogs you want.

One way to fix this without an algorithm is to look at overlap between readers. If many people who follow blog A also follow blog B, B is probably worth a look. That only needs a shared server that counts which feeds are followed together, with no accounts or tracking.

Build a feed reader that lets people follow blogs, read full articles even offline, and discover new blogs through anonymous recommendations.

Tasks

A) Build the Reader

Create a single full-stack web application with two layers:

- **Client**
  - A web UI with a unified timeline, a subscriptions list, a reading view and a Discover page.
  - All feed parsing, article extraction and caching happen in the browser.
- **Server (same app)**
  - **Fetch route:** takes a URL and returns the raw response, since browsers can't fetch most feeds or pages directly because of CORS. It must not parse or store content.
  - **Discovery API:** a database-backed service that collects anonymous subscription data, ranks feeds and serves recommendations.

B) Client-Side Feed Handling

- **Autodiscovery:** Given a site URL, find its feed via `<link rel="alternate">` or common paths such as `/feed` and `/rss.xml`.
- **Parsing:** Support RSS and Atom, and show posts in one chronological timeline.
- **Reading state:** Mark posts read or unread, and filter by feed.
- **Resilience:** Handle dead feeds, malformed XML and slow servers without breaking the timeline.

C) Full-Text Reading

- Fetch the original article through the fetch route, extract the main content (e.g., Mozilla Readability) and show it in a clean reading view.
- **Sanitize** all extracted HTML before rendering (e.g., DOMPurify). Unsanitized proxied content is a security failure.
- Fall back gracefully to the feed's own summary when extraction fails.

D) Anonymous Discovery (Core Technical Challenge)

- **No accounts, no persistent IDs.** Sharing is opt-in. When enabled, the client submits its subscription list as a single anonymous batch.
- **Ranking** combines popularity (how many clients follow a feed) and co-subscription (feeds frequently followed together with the user's own). A simple database query is enough.
- **API:** submit subscriptions, get recommendations for a given set of feeds, and get popular feeds. Recommendations never include feeds the user already follows.
- **Integrity:**
    - Normalize URLs so one blog under several URLs counts once.
    - Validate input: cap batch size, accept only valid `http(s)` URLs, and reject private or local IP ranges.
    - Protect the submission endpoint with at least one abuse-resistance measure (e.g., signed single-use tokens, proof-of-work or rate limiting). Any IP data must be stored only as a salted hash with a short expiry.
    - Document a short threat model in the README: what you defend against and what you accept.

E) Offline-First Reading

- Fetched posts and extracted articles are cached locally (IndexedDB or localStorage).
- The cached timeline loads instantly, with a background refresh when online.
- Read state and cached articles persist across sessions.

Technical Requirements:

- **Stack:** Any language and framework
- **Hosting:** Vercel, Netlify or any platform (deployment is a must)
- **Database:** Any (free tiers such as Supabase or Turso are fine)
- **Libraries:** Feed parsers and Readability-style extractors are allowed
- **Sample Dataset:** A seed list of 10–15 blogs, plus a pre-populated discovery dataset so Discover isn't empty on demo day. Two hidden test sites will be used for evaluation.

Deliverables:

- Complete application code (full-stack app)
- Deployed web application
- Public GitHub repository
- Short demo video (2–3 min): add feeds, read an extracted article offline, and subscribe to a recommendation
- Short presentation for the offline evaluation

Documentation (README.md must include):

1. **Architecture Overview:** diagram of the client, server routes and database
2. **Setup Instructions:** local setup and environment variables
3. **Privacy & Threat Model:** what the server stores and never stores, how opt-in works, and what abuse protection you implemented
4. **Demo Instructions:** sample sites to try and edge cases (no feed, dead feed, extraction failure, duplicate URLs)

Bonus Points:

- **Bonus Quest (10 extra points):** The seed list contains one deliberate trap (two URLs for the same blog, or a malformed feed). The first team to detect it and report it to the mentors earns 10 extra points.
- Transparent recommendations, showing why each one was suggested (e.g., “followed by many readers of X”)
- Additional abuse protection layers (ranking safeguards, minimum distinct submissions, downweighting suspicious batches)
- Background refresh of cached feeds when online
- Blogroll detection, submitted as “A links to B” edges that feed the ranking
- OPML import and export
- Conditional fetching (ETag and Last-Modified) and `robots.txt` support
- A “blogs like this” graph view
- Per-feed folders or tags

PS-004
PrivacyLens: Building a Privacy-Preserving AI Browser Agent

Background

AI-powered agents can automate complex tasks such as shopping, filling forms, managing accounts, and navigating websites. However, these agents often send screenshots and webpage data to cloud-based AI models, potentially exposing sensitive information such as Aadhaar numbers, PAN, passwords, OTPs, financial details, API keys, medical records, and personal messages.

This creates significant privacy and security risks.

Your challenge is to build PrivacyLens, a system that enables AI agents to interact with digital interfaces intelligently while preventing sensitive user information from being exposed to external AI models.

The core principle is:

Compute what is sensitive locally. Reason globally only on sanitized data.

Tasks

A) Sensitive Information Detection

Build a system that identifies sensitive information using one or more techniques:

- **Pattern Detection:** Use regular expressions and validation algorithms to detect Aadhaar, PAN, emails, phone numbers, credit/debit cards, passwords, OTPs, API keys, and other sensitive data.
- **Structural Analysis:** Inspect DOM elements, accessibility trees, metadata, or other interface representations to identify sensitive fields.
- **AI-Based Detection:** Optionally use OCR, computer vision, face detection, or Named Entity Recognition to identify contextual PII.
- **Defense in Depth:** Combine multiple techniques to reduce missed detections and false positives.

B) Privacy-Preserving Sanitization

- Redact sensitive regions in screenshots using blackout, masking, or other suitable techniques.
- Replace sensitive text with semantic placeholders such as `[REDACTED:CARD]` or `[REDACTED:PASSWORD]`.
- Preserve non-sensitive information and interface structure required for AI reasoning.
- Ensure that original sensitive values cannot be recovered from the sanitized data sent externally.

C) AI Reasoning and Automation

- Integrate an LLM, VLM, or another suitable AI model to reason over sanitized information.
- Enable the AI to understand the interface and perform tasks through suitable interaction mechanisms.
- Validate proposed actions and reject invalid targets or unsafe operations.
- Prevent the model from attempting to infer or reconstruct redacted information.
- Require user approval for sensitive or irreversible actions where appropriate.

D) Privacy and Security Validation

- Validate outgoing data before transmission and block requests containing detected, unredacted sensitive information.
- Implement fail-closed behavior when sanitization or validation fails.
- Protect against prompt injection, invalid actions, and accidental data leakage through logs or metadata.
- Provide a before/after privacy preview and an offline or local-only mode where feasible.
- Minimize data retention and clearly define the system's trust boundaries.

Technical Requirements

Technology Flexibility

Participants are free to choose **any technology, architecture, programming language, framework, or tech stack**. Building a browser extension is not mandatory.

Possible approaches include browser extensions, desktop applications, web applications, middleware, local proxies, AI agent frameworks, or hybrid edge-cloud systems.

Teams may use existing tools, libraries, models, and automation frameworks. The choice of technology is open-ended; the solution must demonstrate effective and measurable privacy protection.

Evaluation Scenarios

Test the solution using controlled webpages or equivalent datasets covering:

1. E-commerce checkout — card details and addresses.
2. Authentication — passwords, emails, and OTPs.
3. Customer support — phone numbers and private messages.
4. Video conferencing — faces and participant information.
5. Developer settings — API keys, tokens, and IP addresses.

Use synthetic sensitive data and annotated test cases to measure system performance.

Evaluation Criteria

- **PII Detection Recall:** How much sensitive information is detected?
- **Redaction Precision:** How accurately is sensitive information identified without excessive redaction?
- **Privacy Leakage:** Does sensitive information appear in outgoing payloads?
- **Context Preservation:** Can the AI still understand the interface after sanitization?
- **Task Success:** Can the system complete its intended task?
- **Performance:** What are the processing latency and resource requirements?
- **Security:** How effectively does the system handle adversarial inputs and validation failures?

All reported results must be supported by reproducible tests. Zero leakage claims must be limited to the tested scenarios and transmission pathways.

Deliverables

- Complete source code and a working demonstration.
- Sensitive-information detection and sanitization pipeline.
- AI reasoning and interaction components.
- Test cases, evaluation scripts, and measured results.
- Setup instructions and configuration files.
- A **2–3 minute demo video** showing sensitive-data detection, sanitization, AI task execution, and blocking of an unsafe outgoing payload.

Documentation (README.md)

Include:

1. Architecture diagram and data flow.
2. Setup instructions and configuration.

3. Detection and sanitization methodology.
4. AI integration and action validation.
5. Security assumptions and known limitations.
6. Evaluation methodology and reproducible results.

Bonus Points

- Fully local AI processing without cloud requests.
- Advanced contextual PII detection.
- Adaptive redaction that preserves maximum useful context.
- Protection against prompt injection and adversarial webpages.
- User-defined privacy rules and explainable redaction.
- Local secret injection without exposing values to cloud models.
- Automated leakage testing using synthetic canary values.
- Cross-browser or desktop-agent support.
- Human approval for sensitive actions.

Expected Outcome

Build a functional system that demonstrates how AI agents can interact with digital interfaces while protecting sensitive user information.

The goal is not to build a specific application or use a prescribed tech stack, but to develop an innovative, reliable, and measurable solution to privacy-preserving AI automation.